

TALLER – LABORATORIO PRO

Gestión Profesional de Bases de Datos

Guía paso a paso para estudiantes

Nivel: Avanzado / Profesional

Duración sugerida: 8–12 horas

Tecnología principal: PostgreSQL

Arquitectura: 3-tier

Modalidad: equipos de 2–3 estudiantes

Propósito: construir, asegurar, optimizar, auditar y recuperar una base de datos empresarial, demostrando decisiones técnicas con evidencia.

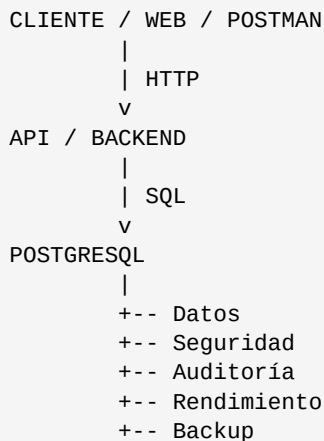
1. Contexto del laboratorio

Ustedes forman parte del equipo DBA de **DataMarket S.A.S.**, una empresa de comercio electrónico. Deben implementar la base de datos de su plataforma de pedidos.

El sistema debe administrar clientes, productos, categorías, inventario, pedidos, detalle de pedidos, pagos, usuarios, roles y auditoría.

El objetivo no es solamente crear tablas. Deben demostrar que la solución es consistente, segura, transaccional, eficiente, auditable y recuperable.

2. Arquitectura objetivo



Trabajarán con una arquitectura de tres capas: presentación, lógica de negocio y persistencia.

3. Resultado esperado

Al terminar deben tener un proyecto organizado, ejecutable y documentado con modelo de datos, scripts SQL, seguridad, transacciones, concurrencia, índices, vistas, funciones/procedimientos, triggers, auditoría, backup, restauración y una API básica.

4. Preparación del entorno

4.1 Instalar herramientas

Instalen Docker Desktop, Git, un editor como VS Code y una herramienta para conectarse a PostgreSQL. Se recomienda trabajar con PostgreSQL mediante Docker para que todos los equipos tengan el mismo entorno.

4.2 Crear la estructura

```
laboratorio-dba/
■■■ README.md
■■■ docs/
■■■ database/
■■■ api/
■■■ tests/
■■■ backup/
■■■ docker/
■■■ screenshots/
```

4.3 Crear Docker Compose

```
services:
  postgres:
    image: postgres:16
    container_name: datamarket-postgres
    environment:
      POSTGRES_DB: datamarket
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

Guarden el archivo como **docker/docker-compose.yml**.

```
docker compose -f docker/docker-compose.yml up -d
docker ps
```

Comprueben que el contenedor de PostgreSQL esté ejecutándose.

5. Paso 1 – Diseñar el modelo

Antes de escribir SQL, transformen los requerimientos en entidades y relaciones.

Entidades mínimas: clientes, categorías, productos, inventario, pedidos, detalle_pedido, pagos, usuarios y auditoría.

Definan para cada entidad: atributos, clave primaria, claves foráneas, cardinalidades y reglas de negocio.

Actividad

1. Construyan el modelo conceptual.
2. Transformen el modelo conceptual en modelo lógico.
3. Definan tipos de datos y restricciones para el modelo físico.
4. Elaboren un diccionario de datos.

Guarden las evidencias en **docs/**.

6. Paso 2 – Crear la base de datos

Organicen los scripts para poder reconstruir la base desde cero.

```
database/  
■■■ 01_database.sql  
■■■ 02_schema.sql  
■■■ 03_tables.sql  
■■■ 04_constraints.sql  
■■■ 05_indexes.sql  
■■■ 06_views.sql  
■■■ 07_functions.sql  
■■■ 08_procedures.sql  
■■■ 09_triggers.sql  
■■■ 10_roles.sql  
■■■ 11_seed.sql
```

Ejecuten los scripts en orden. Verifiquen después de cada etapa que la estructura creada coincide con el diseño.

7. Paso 3 – Tablas e integridad

Implementen las tablas y las restricciones. No permitan que las reglas importantes dependan exclusivamente de la aplicación.

```
CREATE TABLE categorias (  
    id BIGSERIAL PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL UNIQUE  
);
```

Agreguen restricciones PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK y DEFAULT donde corresponda.

Prueben operaciones inválidas: insertar un pedido con un cliente inexistente, un precio negativo o duplicar un dato que deba ser único.

8. Paso 4 – Datos de prueba

Construyan datos suficientes para que las consultas de rendimiento sean significativas. Incluyan clientes, productos, inventario y pedidos de prueba.

Para las pruebas de rendimiento generen un volumen mayor de datos y documenten cómo lo generaron.

9. Paso 5 – Seguridad y roles

Creen roles con responsabilidades diferentes:

```
db_admin
db_developer
db_analyst
db_operator
db_auditor
```

Definan privilegios mínimos. Un analista debe poder consultar información autorizada, pero no eliminar registros. Un auditor debe consultar la información de auditoría sin administrar la base.

Prueben explícitamente permisos permitidos y denegados. Registren capturas de pantalla como evidencia.

10. Paso 6 – Transacciones

Implementen el proceso de compra como una única unidad lógica: crear pedido, insertar detalle, descontar inventario y registrar pago.

```
BEGIN;

-- crear pedido
-- insertar detalle
-- actualizar inventario
-- registrar pago

COMMIT;
```

Simulen un error antes del COMMIT y ejecuten ROLLBACK. Comprueben que no quedan datos parcialmente registrados.

Expliquen qué operaciones deben ser atómicas y por qué.

11. Paso 7 – Concurrency

Simulen dos sesiones intentando comprar simultáneamente un producto cuyo stock sea 1.

Investiguen y prueben los niveles de aislamiento READ COMMITTED, REPEATABLE READ y SERIALIZABLE.

Documenten qué ocurre cuando dos transacciones leen o modifican el mismo recurso y cómo evitarían una inconsistencia.

12. Paso 8 – Optimización

Seleccionen al menos tres consultas relevantes y mídalas con EXPLAIN ANALYZE.

```
EXPLAIN ANALYZE
SELECT *
FROM pedidos
WHERE cliente_id = 1000;
```

Analicen el plan de ejecución. Identifiquen scans, índices, costos y tiempos.

```
CREATE INDEX idx_pedidos_cliente
ON pedidos(cliente_id);
```

Vuelvan a medir y comparen. Cada índice debe tener una justificación técnica.

13. Paso 9 – Vistas

Construyan vistas para simplificar el acceso a información de negocio y limitar la exposición directa de tablas.

Como mínimo creen: resumen de ventas por período, inventario bajo y resumen de compras por cliente.

```
CREATE VIEW inventario_bajo AS
SELECT *
FROM inventario
WHERE stock < stock_minimo;
```

Prueben que los roles adecuados puedan consultar las vistas.

14. Paso 10 – Funciones y procedimientos

Identifiquen operaciones de negocio que puedan centralizarse en PostgreSQL. Definan parámetros, validaciones, resultados y manejo de errores.

Documenten por qué una lógica pertenece a la base de datos y no solamente a la API.

15. Paso 11 – Auditoría

La empresa necesita conocer quién modificó datos críticos, cuándo y qué cambió.

```
CREATE TABLE auditoria (
    id BIGSERIAL PRIMARY KEY,
    usuario TEXT,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    operacion TEXT,
    tabla_afectada TEXT,
    registro_id BIGINT,
    datos_anteriores JSONB,
    datos_nuevos JSONB
);
```

Implementen triggers para operaciones críticas. Realicen UPDATE y DELETE de prueba y comprueben que la auditoría registra el evento.

16. Paso 12 – Backup

Realicen un respaldo de la base de datos.

```
pg_dump -h localhost -p 5432 -U postgres -d datamarket -F c -f backup/datamarket.backup
```

Verifiquen que el archivo exista y documenten fecha, tamaño y procedimiento.

17. Paso 13 – Recuperación

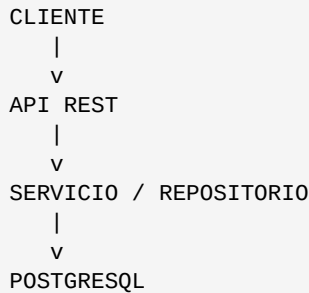
Simulen un incidente controlado sobre una base experimental. Después, restauren el respaldo y comprueben que los datos vuelven a estar disponibles.

```
pg_restore -h localhost -p 5432 -U postgres -d datamarket_restaurada backup/datamarket.backup
```

Documenten el procedimiento y las comprobaciones realizadas después de la restauración.

18. Paso 14 – Integración con una API

Construyan una API sencilla en Node.js o Python que consuma PostgreSQL.



Como mínimo implementen operaciones para consultar productos, consultar inventario, crear un pedido y consultar pedidos.

La API no debe conectarse utilizando el usuario administrador. Utilicen privilegios mínimos.

19. Paso 15 – Pruebas

Construyan pruebas para comprobar integridad referencial, restricciones, permisos, transacciones, concurrencia, rendimiento, auditoría y backup/restore.

Cada prueba debe indicar: objetivo, preparación, acción, resultado esperado y resultado obtenido.

20. Reto final – Incidente de producción

El docente les entregará o activará un escenario con problemas intencionales. Su misión será diagnosticar y corregir el sistema.

Ejemplos: permisos excesivos, consulta lenta, inconsistencia de inventario, tabla sin auditoría, índice ausente o backup que no puede restaurarse.

No se limiten a corregir el problema. Demuestren cómo lo detectaron, qué evidencia obtuvieron, qué cambio realizaron y cómo comprobaron la solución.

21. Entrega final

```
laboratorio-dba/
■■■ README.md
■■■ docs/
■   ■■■ arquitectura.md
■   ■■■ decisiones-tecnicas.md
■   ■■■ diccionario-datos.md
■■■ database/
■   ■■■ 01_database.sql
■   ■■■ 02_schema.sql
■   ■■■ 03_tables.sql
■   ■■■ 04_constraints.sql
■   ■■■ 05_indexes.sql
■   ■■■ 06_views.sql
■   ■■■ 07_functions.sql
■   ■■■ 08_procedures.sql
■   ■■■ 09_triggers.sql
■   ■■■ 10_roles.sql
```

```
■ ■■■ 11_seed.sql
■ ■■■ api/
■ ■■■ tests/
■ ■■■ backup/
■ ■■■ docker/
■ ■■■ screenshots/
```

22. Checklist del estudiante

- El proyecto puede levantarse desde cero.
- La base puede reconstruirse ejecutando los scripts en orden.
- Las restricciones impiden datos inválidos.
- Los roles tienen privilegios diferenciados.
- Las operaciones críticas utilizan transacciones.
- Se probó concurrencia.
- Se analizaron consultas con EXPLAIN ANALYZE.
- Los índices tienen justificación.
- Existen vistas de negocio.
- Existe auditoría.
- Se generó un backup.
- Se realizó una restauración.
- La API utiliza privilegios mínimos.
- Las pruebas tienen evidencia.
- El README explica cómo ejecutar todo.

23. Preguntas de reflexión técnica

1. ¿Qué reglas deben garantizarse en la base y cuáles en la aplicación?
2. ¿Qué problema de concurrencia encontraron y cómo lo resolvieron?
3. ¿Qué consulta presentó mayor costo y por qué?
4. ¿Por qué eligieron cada índice?
5. ¿Qué usuario debería tener acceso a cada objeto?
6. ¿Qué operaciones deben ser auditadas?
7. ¿Qué ocurriría si se pierde el servidor de base de datos?
8. ¿Cuánto tiempo necesitan para recuperar el servicio con el backup realizado?
9. ¿Qué decisiones cambiarían si el sistema pasara de 1.000 a 10 millones de registros?
10. ¿Qué decisiones de diseño afectan más el rendimiento y la seguridad?

Fin del laboratorio. El criterio de trabajo es demostrar cada decisión mediante ejecución, evidencia y explicación técnica.